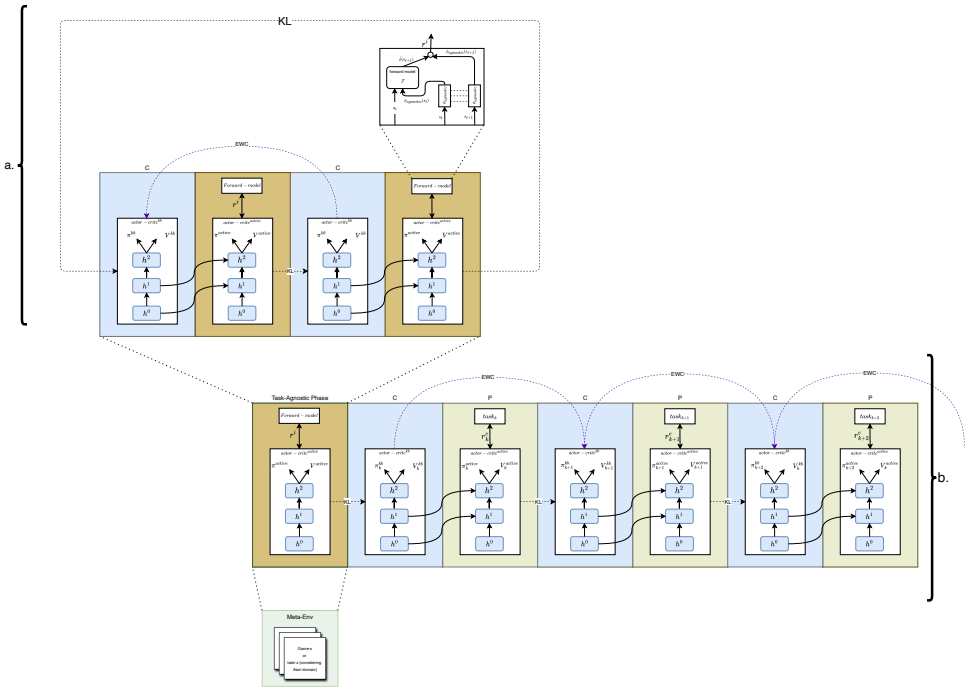

Fig. 2: Overview of our Task-Agnostic Policy Distillation framework. (a) The task-agnostic phase is an abstraction of a process where intermediate alternations between maximizing intrinsic rewards and distillation occur. This process follows the same alternating pattern as in the progress and compress framework. (b) Here, the task-agnostic phase is initially used before alternating between progress (P) and compress (C) phases. When considering the Atari domain, each task can be randomly selected from the Meta-Environment, therefore simulating one game environment. In the C phase, the recently learned policy by the active column (green) is distilled into the knowledge base (KB) (blue) using the KL loss between the active column and KB while protecting KB's old values using Elastic Weight Consolidation (EWC). In the P phase, features learned from previous tasks are reused via lateral connections when learning new tasks. r_k^e and r^i are the extrinsic reward of task k and the task-independent intrinsic reward, respectively. h is a hidden layer.

Variation 2. The second variation relies on configuring the general Variation 1 of the task-agnostic phase. This means configuring the Meta-Environment in such a way that it only has one task (i.e., Atari game environment) to explore. Therefore, within the task-agnostic phase, only one game environment is being distilled in alternating phases. After each intermediate timestep, the distillation process begins again, followed by exploration, as described in Sequence 4, but without selecting different game environments after each intermediate step. This reflects the case where we only have one environment with different tasks, unlike the Atari domain where one environment has only one task, such as scoring as many points as possible.

Variation 3. The third variation relies on configuring Variation 2 of the task-agnostic phase. This involves setting the number of timesteps to only run for a specific number of steps and compressing the task-agnostic policy only once, before adhering to the standard protocol of the Progress and Compress method, proposed by [13]. This follows the steps (the downstream task loss $\mathcal{L}^{progress}(\theta)$ is given in Appendix A):

$$\xrightarrow[x-steps]{\text{explore/train}} \min \mathcal{L}^{agnostic}(\theta) \xrightarrow[x-steps]{\text{compress}} \theta^{kb} \xrightarrow[y-steps]{\text{train}} \min \mathcal{L}^{progress}(\theta) \xrightarrow[y-steps]{\text{compress}} \dots \quad (5)$$

This approach relaxes the requirement for alternating distillation of task-agnostic policies, as only one task-agnostic policy would be distilled into the knowledge base. An analogy can be drawn with an agent that explores one room, acquiring skills within it that can later be applied to different rooms.

4.3 Intrinsic Reward

The task-agnostic agent predominantly utilizes self-supervised prediction as its learning mechanism. By learning to predict future states from current states and actions, the agent is able to navigate and comprehend its environment, eliminating the need for extrinsic rewards. This intrinsic motivation, inspired by the ICM module of Pathak et al. [7], is driven by the prediction error of a forward dynamics model and enables the agent to explore and acquire knowledge about its surroundings, preparing it for future tasks in diverse environments. A distinction from Pathak et al. [7] is made by excluding the inverse dynamics model of the ICM module, as this work is solely focused on learning a forward model. In alignment with [7], it is advantageous to make predictions about the next state within the feature space. Hence, let $\phi_{agnostic}$ be the visual encoder (here, a convolutional neural network) and let $\mathcal{F}$ be a multi-layer perceptron. If the state at timestep t is represented as s_t (raw pixels) and the action as a_t, then $\mathcal{F}(\phi_{agnostic}(s_t), a_t) = \hat{\phi}(s_{t+1})$, where $\hat{\phi}(s_{t+1})$ is the predicted feature vector of the next state. Therefore, the composition of the networks $\mathcal{F}$ and $\phi_{agnostic}$ forms the ICM module, excluding the inverse model. The loss used to optimize the forward dynamic model is defined as an L_2 norm of the difference between the observed and predicted feature vectors of s_{t+1} i.e. $\mathcal{L}^{forward} = \left\| \phi_{agnostic}(s_{t+1}) - \hat{\phi}(s_{t+1}) \right\|_2^2$. The overall intrinsic reward used to seek novel states is $r_t^i = \log(\mathcal{L}^{forward} + \epsilon)$, where ϵ is a constant added to maintain numerical stability, for values near zero.

Using the error in predicting future states as an intrinsic reward results in exploratory behavior that enables the agent to explore and acquire knowledge about its surroundings without the need for external supervisory signals, relying only on its curiosity. By encouraging visits to states that are difficult to predict (high prediction error areas), this approach fosters a directed exploration strategy aimed at improving the agent's forward model. This strategy is particularly effective in sparse-reward environments, as it does not depend on extrinsic rewards. During the task-agnostic phase, we apply this principle to learn an exploratory policy that is periodically distilled into a knowledge base. After each distillation, a new game environment is selected where the agent further refines its task-agnostic policy using intrinsic rewards and lateral connections from the knowledge base, which now includes the recently distilled policy.

This new policy is once again distilled and used to enhance the knowledge base. The distillation of the exploratory action distribution of one policy into another augments and diversifies the exploratory behavior. After the task-agnostic phase, learning downstream tasks in the subsequent progress and compress phases is accelerated due to the generalized knowledge from task-agnostic exploration stored in the knowledge base.

5 Experimental Evaluation

In this section, we present the experimental evaluation of our approach. First, Section 5.1 describes the Atari games that constitute the Meta-Environment to which our approach and the compared methods are applied. Next, Section 5.2 details the implementation of the task-agnostic phase and demonstrates the agent's performance and learning progress during this phase. Then, Section 5.3 compares the performance of all methods on downstream tasks in terms of learning efficiency assessed using the average game score and the entropy of the policy distribution during the progress phase, where tasks are encountered sequentially. Following this, Section 5.4 evaluates forward transfer as an indicator of adaptability by analyzing the average score on each subsequent visit to the considered tasks. Finally, Section 5.5 evaluates the computational efficiency of each method, highlighting the trade-offs between performance and resource demands.

5.1 Environments

As in Schwarz et al. [13], the task-agnostic policy distillation framework is used in the Atari domain. In this paper, we focuses on five different Atari games: Pong, SpaceInvaders, BeamRider, DemonAttack, and AirRaid. The Meta-Environment combines these five game environments. To modify the action space, a custom action wrapper is employed. This wrapper maps the intended actions of the agent to the corresponding movements within the specific game environment. The action space is downscaled, removing unnecessary actions, as suggested by [43]. Table 1 illustrates how actions are mapped to different games/tasks. This mapping can vary depending on the specific requirements of each game/task. For instance, in the case of BeamRider, the agent selects action 2 from the policy distribution. However, the game interface executes action 3, as specified in BeamRider. As a result, the actual (game-specific) action space is hidden from the agent.

Table 1: Action mapping for different games/tasks

Game/Task	NOOP (0)	FIRE (1)	RIGHT (2)	LEFT (3)
Pong	0	1	2	3
BeamRider	0	1	3	4
SpaceInvaders, DemonAttack, AirRaid	0	1	2	3

These modifications to the action space prevent the attempt of "pointless" actions. By reducing the size of the action space, the complexity of the problem is decreased, requiring fewer computational resources and potentially speeding up training time.